



به نام خدا



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر
مدل‌های مولد عمیق
دکتر توسلی‌پور
تمرین دوم

نام و نام خانوادگی	علیرضا نجفی مطیعی
شماره دانشجویی	۸۱۰۱۰۰۲۲۴
تاریخ ارسال گزارش	۱۴۰۴.۰۹.۰۵

فهرست

۳		سؤال اول: Normalizing Flow
۳		بخش اول: بخش تئوری
۴		بخش دوم: بخش عملی
۱۰		سوال دوم: CycleGAN
۱۰		بخش اول: سوالات تئوری
۱۴		بخش دوم: پیاده سازی مدل
۲۶		مراجع

سؤال اول: Normalizing Flow

بخش اول: بخش تئوری

زیربخش اول:

۱.

$$f_Z(z) = \begin{cases} 1 & 0 \leq z \leq 1 \\ 0 & \text{o.th} \end{cases}$$

$$Z = \frac{x-1}{2}, \frac{d}{dx} \frac{x-1}{2} = \frac{1}{2}, f_X(x) = f_Z\left(\frac{x-1}{2}\right) \cdot \left|\frac{1}{2}\right|$$

$$\rightarrow P_X(x) = \begin{cases} \frac{1}{2} & 1 \leq x \leq 3 \\ 0 & \text{o.th} \end{cases}$$

۲.

$$f_Z(z) = \begin{cases} \frac{1}{2} & 0 \leq z \leq 2 \\ 0 & \text{o.th} \end{cases}$$

$$Z = \ln(x), \frac{d}{dx} \ln(x) = \frac{1}{x}, f_X(x) = f_Z(\ln(x)) \cdot \left|\frac{1}{x}\right|$$

$$\rightarrow P_X(x) = \begin{cases} \frac{1}{2x} & 1 \leq x \leq e^2 \\ 0 & \text{o.th} \end{cases}$$

زیربخش دوم:

۱.

$$z_1 = x_1, z_2 = x_2 + m(x_1)$$

$$\frac{\partial x_1}{\partial z_1} = 1, \frac{\partial x_1}{\partial z_2} = 0, \frac{\partial x_2}{\partial z_1} = m'(x_1), \frac{\partial x_2}{\partial z_2} = 1$$

$$\rightarrow J = \begin{bmatrix} I & 0 \\ m'(x_1) & I \end{bmatrix} \rightarrow \det(J) = I \cdot I = 1$$

۲.

$$z_1 = x_1, z_2 = s(x_1)x_2 + t(x_1) \rightarrow \frac{z_2 - t(z_1)}{s(z_1)} = x_2$$

$$\frac{\partial x_1}{\partial z_1} = 1, \frac{\partial x_1}{\partial z_2} = 0, \frac{\partial x_2}{\partial z_1} = \frac{(t(z_1) - z_2)s'(z_1) - t'(z_1)s(z_1)}{s^2(z_1)}, \frac{\partial x_2}{\partial z_2} = \frac{1}{s(z_1)}$$

$$\rightarrow \det(J) = \frac{1}{s(z_1)}$$

بخش دوم: بخش عملی

زیربخش اول:

به پیاده‌سازی مدل MAF طبق معماری MADE پیشنهادی و سایر پارامترهای داده‌شده در صورت پروژه بدون تغییر دادن آن‌ها می‌پردازیم.

MADE(Masked Autoencoder for Distribution Estimation): پیاده‌سازی MADE یک هدف ساده را دنبال می‌کند و به دنبال این است تا همزمان با استفاده از یک شبکه dense، Autoregressive Property را هم حفظ کنیم. یعنی پیکسل شماره ۲ تنها پیکسل شماره یک را ببیند، پیکسل شماره ۳ تنها پیکسل‌های ۱ و ۲ و به همین ترتیب. در حالت عادی در یک شبکه dense یک لایه Linear همه ورودی‌ها را به همه خروجی‌ها وصل می‌کند. MADE با استفاده از تکنیک Masking اتصالات غیر لازم را حذف می‌کند. برای اینکه متوجه شویم کدام اتصال باید قطع شود یا به عبارت دیگر در ماتریس وزن‌ها صفر شود، به هر نورون یک عدد یا درجه نسبت می‌دهیم. به هر پیکسل ورودی به ترتیب از یک تا تعداد تعداد کل پیکسل‌ها D عدد داده می‌شود. به هر نورون در لایه مخفی نیز یک عدد تصادفی نسبت داده می‌شود با شرط اینکه این عدد باید بین مینیمم درجه ورودی‌ها و تعداد کل پیکسل‌ها منهای یک (D-1) باشد. در خروجی نیز دقیقاً همان عدد ورودی متناظر خود را داریم. پس از اینکه همه نورون‌ها درجه‌دار شدند باید ماتریس‌های وزن را با رعایت یکسری قوانین در یک ماتریس باینری Mask ضرب کنیم:

لایه اول (از ورودی به لایه مخفی): یک نورون مخفی با درجه k تنها می‌تواند از نورون‌های ورودی با درجه d سیگنال بگیرد اگر $d \leq k$ باشد.

لایه دوم (از لایه مخفی به خروجی): یک نورون خروجی با درجه d تنها می‌تواند از نورون‌های مخفی با درجه k سیگنال بگیرد اگر $k < d$ باشد.

بلاک MAF(Masked Autoregressive Flow): این مدل درواقع از MADE به عنوان یک تابع برای تولید پارامترهای توزیع نرمال استفاده می‌کند. می‌دانیم که رابطه زیر در این مدل برقرار است:

$$z_i = \frac{x_i - \mu_i}{e^{\alpha_i}}$$

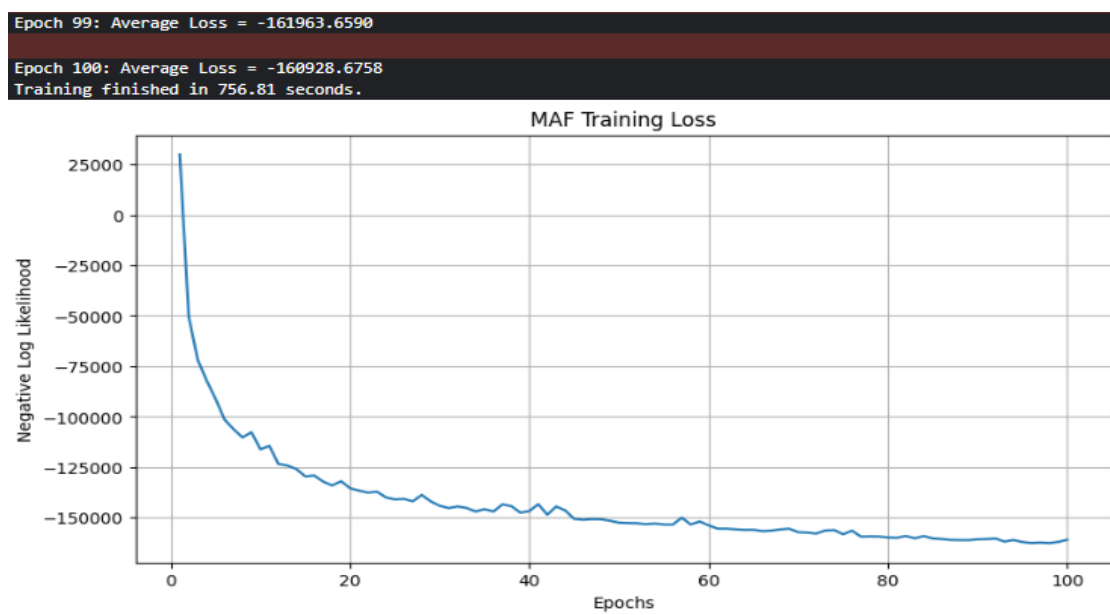
که μ_i (میانگین) و α_i (لگاریتم انحراف معیار) خروجی‌های شبکه MADE هستند.

در ابتدا کل بردار تصویر که Flattened شده وارد MADE می‌شود. شبکه MADE با توجه به ماسک‌ها برای هر پیکسل i دو عدد μ_i و α_i تولید می‌کند. به دلیل استفاده از این ماسک‌ها μ_i و α_i تنها تابعی از پیکسل‌های قبلی خواهند بود. پس از آن فرمول ذکر شده در بالا استفاده می‌شود تا x به z تبدیل شود. سپس باید دترمینان ماتریس ژاکوبین را به دست آوریم که با توجه به اقدامات قبلی به جهت Autoregressive ماندن تبدیل با کمک ماسک‌ها یک ماتریس پایین‌مثلثی است بنابراین دترمینان آن برابر است با حاصل ضرب درایه‌های قطر اصلی. بنابراین Log دترمینان برابر می‌شود با $\sum(-\alpha_i)$.

یکی دیگر از نکات پیاده‌سازی در اینجا این است که در این مسئله تعداد نورون‌های مخفی یا k از ورودی که $3 \times 128 \times 128$ است یعنی d بسیار کوچک‌تر است. در این حالت به تعداد k از d عدد تصادفی انتخاب می‌کنیم به این صورت که هر نورون انتخاب شده پردازش جزئیات یک بازه را بر عهده می‌گیرد. اما باید توجه شود که اعداد تصادفی به خوبی انتهاب شوند تا نمونه‌ها به طور تقریباً مساوی در کل بازه پخش شوند. همچنین دلیل استفاده از ۷ بلوک MADE این است یک بلوک تنها نمی‌تواند پیچیدگی‌ها را پردازش کند و ترکیب این چند ترکیب ساده MADE می‌تواند توزیع احتمالی پیچیده‌تری را مدل کند. عدد ۷ نیز عدد نه خیلی کم که مدل کلا عمیق نشود و یادگیری درست انجام نشود و نه عدد خیلی بزرگی است تا سرعت آموزش به شدت کند شود.

زیربخش دوم:

به آموزش مدل می‌پردازیم. پاسخ به خواسته‌های صورت پروژه در زیر آمده‌است:



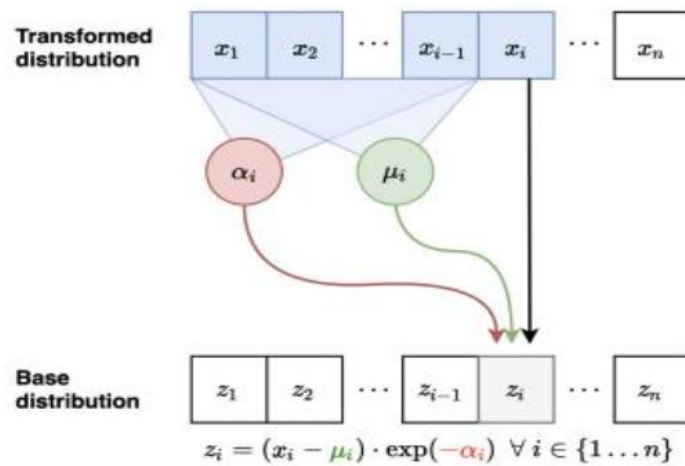
شکل ۱. نمودار خطای مدل در آموزش به همراه مدت زمان آموزش

Sampling finished in 1995.75 seconds.

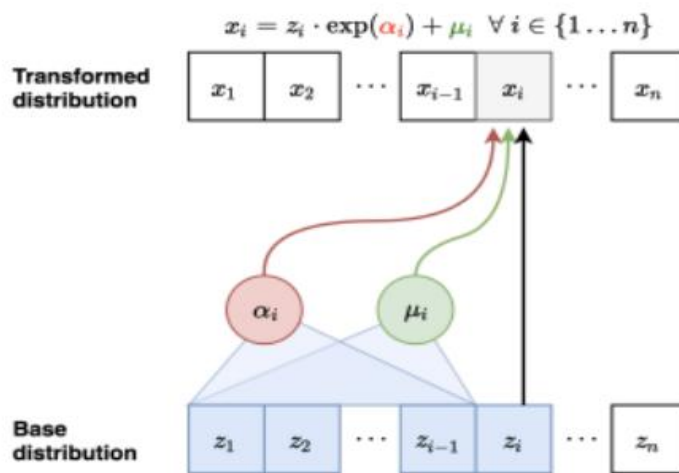


شکل ۲. تصاویر تولید شده به همراه زمان تولید ۵ عکس

۱. مشاهده می‌شود که مدت زمان کل آموزش در حدود ۷۵۶.۸۱ ثانیه و تولید ۵ تصویر در حدود ۱۹۹۵.۷۵ ثانیه طول کشیده است که بسیار بیشتر از زمان آموزش مدل است.
 ۲. مدل‌های Autoregressive بر اساس وابستگی شرطی کار می‌کنند. به عبارت دیگر برای دانکه مدل بتواند پیکسل شماره ۲ را تولید کند باید پیکسل شماره یک را داشته باشد و برای تولید پیکسل ۳ باید ۱ و ۲ را داشته باشد و به همین ترتیب تا آخرین پیکسل را تولید کند. به عبارت دیگر نمی‌توان پردازش را به صورت موازی به انجام رساند و باید تمامی پیکسل‌ها تک به تک تولید شوند و به همین دلیل است که تولید عکس در این مدل‌ها بسیار کند است.
 ۳. در بالا قابل مشاهده است.
 ۴. در مدل‌های MAF پارامترهای μ_i و α_i بر اساس داده‌های قبلی یا همان x های قبلی به دست می‌آیند. برای همین در آموزش این مدل‌ها که تمام نمونه‌های آموزشی را داریم محاسبات می‌تواند به صورت موازی انجام شود و تمام پارامترها را موازی حساب کنیم. اما در تولید نمونه از z چون تمامی x ها در لحظه موجود نیستند و برای محاسبه μ_i و α_i باید صبر کنیم تا x ها مرحله به مرحله تولید شوند نمونه‌برداری این مدل‌ها بسیار طولانی می‌شود.
- اما در مدل IAF پارامترهای μ_i و α_i بر اساس z های قبلی محاسبه می‌شوند و نه داده‌ها. برای همین چون در تولید تصاویر ما بردار z را به طور کامل تشکیل می‌دهیم و آن را از ابتدا داریم می‌توانیم محاسبه پارامترها را به صورت موازی انجام دهیم و نیازی نیست تا صبر کنیم پیکسل‌های قبلی تولید شوند. اما آموزش این مدل برعکس مدل قبلی بسیار کند است چرا که ما در آموزش x ها را داریم و باید صبر کنیم تا z های قبلی محاسبه شوند تا در هر مرحله پارامترها را محاسبه کنیم و دیگر موازی نیست.



شکل ۳. ساختار کلی MAF



شکل ۴. ساختار کلی IAF

زیربخش سوم:

۱. دلیل اصلی عدم استفاده از معیار دقت در anomaly detection نامتوازن بودن داده‌هاست. در اینگونه مسائل معمولاً اکثر داده‌ها (مثلاً ۹۵٪) سالم و درصد کمی از آن‌ها (مثلاً ۵٪) ناهنجار هستند. در این حالت اگر مدل تمامی داده‌ها را سالم تشخیص دهد دقت ۹۵ درصدی خواهد داشت اما نتوانسته هیچ ناهنجاری‌ای را کشف کند و عملاً فاقد ارزش است. برای همین از معیارهایی استفاده می‌شود که به تفکیک‌پذیری کلاس‌ها حساس باشند. یکی از این معیارها AUROC می‌باشد. این معیار نشان می‌دهد که مدل چقدر توانایی دارد که به داده‌های ناهنجار نمره بالاتری نسبت به داده‌های سالم بدهد، فارغ از اینکه آستانه تصمیم‌گیری چقدر است. از دیگر معیارهای ارزیابی AUPR که بر روی کلاسی که ناهنجاری آن مثبت است تمرکز می‌کند. این معیار صحت و بازخوانی را در مقابل یکدیگر رسم

می‌کند و مقدار بالای آن نشان دهنده این است که مدل توانسته بیشتر ناهنجاری‌ها را تشخیص دهد بدون آن که موارد زیادی را که ناهنجار نبوده‌اند ناهنجاری تشخیص دهد. یک معیار دیگر Optimal F1-score است که معیار F1 را که میانگین هارمونیک صحت و بازخوانی است را برای تمامی آستانه‌ها بررسی کرده و بالاترین مقدار آن را گزارش می‌دهد. درواقع می‌گویند که بهترین عملکردی که مدل می‌تواند ارائه دهد اگر پارامتر بهینه را پیدا کنیم در چه حد است.

۲. Anomaly score یک عدد اسکالر است که مدل به هر نمونه داده نسبت می‌دهد و نشان‌دهنده میزان غیرعادی بودن آن است. در مدل‌های احتمالاتی این نمره معمولاً عکس احتمال وقوع داده است. هرچه این عدد بزرگ‌تر باشد یعنی از نظر مدل آن داده شباهت کمتری به داده‌های نرمال دارد و احتمال ناهنجاری آن بیشتر است. درنهایت برای تصمیم‌گیری که آیا ناهنجار است یا خیر این عدد با یک Threshold مقایسه می‌شود.

۳. این مدل یاد می‌گیرد تا توزیع پیچیده دادگان سالم را به یک توزیع ساده و شناخته‌شده مثل نرمال استاندارد ببرد. در هنگام آموزش مدل تنها با تصاویر سالم روبرو می‌شود و یاد می‌گیرد که ویژگی‌های این داده‌ها را به فضای Z ببرد. در هنگام تست وقتی یک تصویر جدید به مدل داده می‌شود مدل تلاش می‌کند تا آن را به فضای Z ببرد و احتمال آن را محاسبه کند. اگر تصویر سالم باشد مدل نمونه مشابه را قبلاً دیده و احتمال بالا به آن نسبت می‌دهد. اگر تصویر ناهنجار باشد مدل نمی‌تواند آن را به خوبی توصیف کند و احتمال پایینی را به آن نسبت خواهد داد. در اینجا از منفی لگاریتم احتمال به عنوان Anomaly score استفاده می‌کنیم.

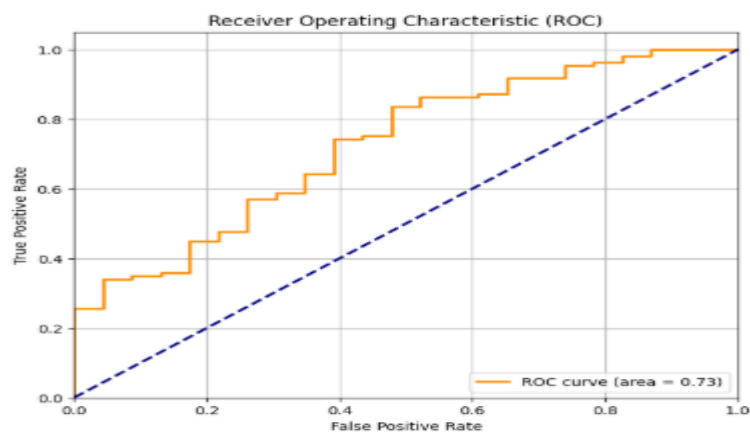
به ارزیابی مدل می‌پردازیم:

(۱)

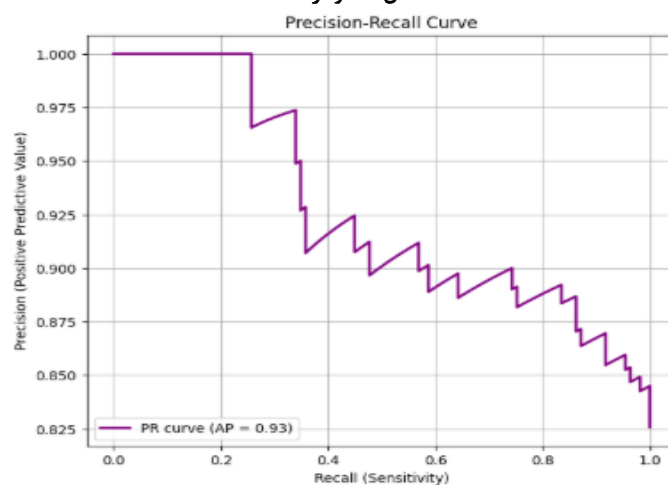
```
Test samples found: 132
Normal count: 23
Anomaly count: 109
Starting Evaluation on Test Set...
Evaluating: 100% | 132/132 [00:11]

=====
Evaluation Result for Linear MAF:
AUROC Score: 72.56%
AUPR Score (Avg Prec): 92.65%
Optimal F1-Score: 91.60%
(Best Threshold: -160942.8906)
=====
```

شکل ۵. نتیجه ارزیابی مدل



شکل ۶. نمودار ROC



شکل ۷. نمودار صحت-بازخوانی

مشاهده می‌شود که طبق خواسته پروژه مقدار AUROC از ۰.۶ بیشتر و برابر با ۰.۷۳ است و مدل تقریباً توانسته است تا اطلاعات کلی تصویر را استخراج کند.

۲) خیر. چرا که مدل‌های Normalizing Flow دارای خاصیت برگشت پذیری هستند و فلوی آن‌ها دوسویه طراحی شده. بنابراین فرقی نمی‌کند که ورودی چیست، در هر صورت خطای بازسازی برای NF چیزی نزدیک به صفر می‌باشد. بنابراین این روش استفاده از NF در Anomaly detection صحیح نیست و روش صحیح روش بخش قبل است.

بخش اول: سوالات تئوری

زیربخش اول: تابع زیان مدل

۱. روش‌های تبدیل تصویر به تصویر مبتنی بر یادگیری نظارتی مانند pix2pix، برای آموزش شبکه عصبی نیاز به داده‌های جفت‌داده‌شده دارند. یعنی برای هر تصویر در دامنه مبدأ، باید دقیقاً تصویر متناظر آن در دامنه مقصد وجود داشته باشد. جمع‌آوری این داده‌های جفت‌شده در دنیای واقعی بسیار دشوار و هزینه‌بر است. علاوه بر آن در برخی حالت‌ها مانند تبدیل اسب به گورخر در واقعیت هیچ داده جفت‌شده‌ای وجود ندارد چرا که ما نمی‌توانیم برای مثال اسب و گورخر را در یک حالت کاملاً یکسان قرار دهیم و این زوج به صورت طبیعی در طبیعت وجود ندارند و نمی‌توان با جمع‌آوری دادگان بیشتر مشکل را حل کرد.

۲. ایده اصلی CycleGAN برای دور زدن این محدودیت داده‌ای استفاده از یادگیری غیرنظارتی با تکیه بر ویژگی برگشت‌پذیری می‌باشد. فرض ما در این مدل این است که اگر یک تصویر را از دامنه x به دامنه y ببریم و سپس نتیجه حاصل را دوباره به دامنه x برگردانیم باید به همان تصویر اولیه برسیم. قید خاص اضافه شده در اینجا Cycle Consistency Loss (زیان سازگاری چرخه) است که درواقع بیان می‌کند که دو مولد تصویر G و F باید معکوس یکدیگر باشند.

$$x \rightarrow G(x) \rightarrow F(G(x)) \approx x, y \rightarrow F(y) \rightarrow G(F(y)) \approx y$$

$$L_{cyc}(F, G) = E_{p_{data}(x)}[||F(G(x)) - x||_1] + E_{p_{data}(y)}[||G(F(y)) - y||_1]$$

۳. تحمیل این قید در مسئله حیاتی است زیرا این مسئله یک مسئله Under-constrained است و اگر تنها بر زیان تخصصی GAN تمرکز کنیم و قید چرخه را حذف کنیم شبکه تنها یاد می‌گیرد که خروجی‌هایی تولید کند که مشابه تصاویر دامنه مقصد باشد اما تضمینی وجود نخواهد داشت که خروجی ربطی به ورودی داشته باشد. یکی از رفتارهای نامطلوبی که می‌تواند رخ دهد Mode Collapse است. شبکه ممکن است تمامی تصاویر ورودی مثلاً تمام اسب‌ها را به تنها یک تصویر خروجی خاص مثلاً یک گورخر خاص که می‌تواند در این حالت تمایزدهنده را به خوبی فریب دهد تبدیل می‌کند. یکی دیگر از رفتارهای نامطلوب می‌تواند عدم حفظ محتوا باشد. یعنی شبکه ممکن است یک مثلاً اسب ایستاده را بگیرد و تصویر یک گورخر در حال دویدن را تولید کند. در این حالت اگرچه تصویر خروجی واقعی به نظر می‌رسد اما تناظر محتوایی با ورودی ندارد. بنابراین قید

سازگاری چرخه شبکه را مجبور می‌کند که ساختار و محتوای اصلی تصویر را حفظ کند و فقط بافت را تغییر دهد تا بتواند دوباره آن را به حالت اولیه بازگرداند.

۴. رابطه کامل تابع زیان به صورت زیر است:

$$L(G, F, D_x, D_y) = L_{GAN}(G, D_y, X, Y) + L_{GAN}(F, D_x, Y, X) + \lambda L_{cyc}(G, F)$$

که در این تابع زیان L_{GAN} تلاش می‌کند تا تصاویر تولیدشده توسط G و F را واقعی جلوه دهد و L_{cyc} نیز مفصلاً توضیح داده شد. نقش λ این است که تعادل بین دو هدف توابع زیان را کنترل می‌کند. زیان GAN به دنبال تغییر تصویر و زیان پایداری چرخه به دنبال حفظ تصویر است. اگر مقدار این ضریب بیش از حد بزرگ انتخاب شود مدل اولویت بسیار زیادی به بازسازی دقیق تصویر اولیه می‌دهد و در نتیجه مدل از ایجاد تغییرات اساسی در تصویر خودداری می‌کند و تصویر خروجی تقریباً همان تصویر ورودی خواهد بود.

۵. زیان همانی به این مفهوم اشاره دارد که اگر تصویری که هم‌اکنون متعلق به دامنه مقصد است را به مولد بدهیم نباید تغییری در آن ایجاد کند. برای مثال اگر مولد G وظیفه تبدیل اسب به گورخر را دارد، اگر به آن به عنوان ورودی یک گورخر بدهیم خروجی باید همان گورخر بدون تغییر باشد.

$$L_{identity}(F, G) = E_{p_{data}(x)}[||F(x) - x||_1] + E_{p_{data}(y)}[||G(y) - y||_1]$$

در مقاله از مثال معروف تبدیل نقاشی‌های مونه به عکس واقعی استفاده شده است. بدوم وجود این زیان مدل ممکن است یاد بگیرد که نه تنها بافت نقاشی را تغییر دهد بلکه تناژ رنگی آن را هم عوض کند. مثلاً اگر نقاشی‌های مونه بیشتر آبی هستند و یک عکس واقعی طیف وسیع‌تری دارد، مدل امکان دارد تا سعی کند رنگبندی عکس‌ها را تغییر دهد مثلاً صبح را به غروب تبدیل کند یا رنگ سفید را به آبی متمایل کند. با وجود قید زیان همانی چون مدل مجبور می‌شود عکس واقعی را بدون تغییر بازتولید کند، یاد خواهد گرفت که ترکیب رنگی ورودی را نیز حفظ کند. به عبارت دیگر این زیان از تغییرات ناخواسته در رنگ و روشنایی جلوگیری می‌کند و باعث می‌شود در تغییر دامنه رنگ‌های اصلی اشیاء حاضر در تصویر تا جای ممکن حفظ شود.

زیربخش دوم: معماری و فرایند آموزش مدل

۱. از منظر تئوری یکی از دلایل می‌تواند تفاوت ماهیتی توابع نگاشت باشد. تابع ریاضی‌ای که ویژگی‌های دامنه x را به ویژگی‌های دامنه y تبدیل می‌کند لزوماً همان تابعی نیست که برعکس آن را انجام می‌دهد. در واقع فضای ویژگی در این دامنه توزیع متفاوتی دارد. بنابراین با یک شبکه عصبی واحد نمی‌تواند همزمان هم مسیر رفت و هم برگشت را بهینه کند چرا که ممکن است هر یک از این

عملیات‌ها به وزن‌ها و فیلترهای متفاوتی نیاز داشته باشند. یک مورد دیگر این است که اگر از یک مولد واحد استفاده کنیم آن مولد باید یاد بگیرد تا معکوس خودش باشد یعنی $G(G(x)) \approx x$ که یک قید بسیار مشکل برای یک شبکه عصبی عمیق است و احتمالاً باعث می‌شود تا شبکه تابع همانی را یاد بگیرد و عملاً تصویر را تغییر را ندهد. از دلایل عملی‌تر می‌تواند این باشد که اساس Cycle GAN بر اساس قید سازگاری چرخه بنا شده و برای این قید ما به دو مولد متمایز نیاز داریم تا بتوانیم خطای رفت و برگشت را جداگانه محاسبه کنیم و گرادیان را به درستی به شبکه منتقل کنیم. همچنین استفاده از دو مولد و دو تمایزگر آموزش پایدارتری را نسبت به اینکه بخواهیم شبکه را مجبور کنیم تا همزمان دو توزیع کاملاً متفاوت را یاد بگیرد، برای ما فراهم می‌کند.

۲. مدل pix2pix از شبکه U-Net استفاده می‌کند که ویژگی بارز معماری آن وجود یکسری Skip Connection بین لایه‌های انکودر و دیکودر است. این مدل روی داده‌های جفت‌شده آموزش می‌بیند و در این حالت ساختار هندسی و لبه‌های تصویر ورودی باید دقیقاً در خروجی حفظ شوند. اتصالات میانبر اجازه می‌دهند تا اطلاعات مکانی و جزئیات مستقیماً از ورودی به خروجی منتقل شوند و از باتلنک شبکه عبور نکنند. بنابراین این مدل برای کارهایی که نیاز به Pixel alignment دارند مناسب است. اما معماری Cycle GAN مبتنی بر ResNet است که شامل یک بخش downsampling، تعداد مشخصی residual block در میانه و پس از آن upsampling است. در اینجا اتصالات میانبر طولانی مانند درون U-Net وجود ندارند. دلیل این انتخاب این است که این مدل برای داده‌های جفت نشده است و اگر از U-Net استفاده شود اتصالات میانبر باعث می‌شدند اطلاعات تصویر ورودی با قدرت به خروجی منتقل شوند و شبکه نمی‌توانست تغییرات عمده‌ای در تصویر ایجاد کند و مثلاً اسب را به گورخر تبدیل کند. معماری ResNet با داشتن باتلنک اطلاعاتی و بلوک‌های باقی‌مانده به شبکه اجازه می‌دهد تا ویژگی‌های عمیق‌تری را یاد بگیرد و تغییرات ساختاری و بافتی پیچیده‌تری را اعمال کند بدون اینکه مجبور شود تا عیناً پیکسل‌های ورودی را کپی کند.

۳. هدف اصلی استفاده از PatchGAN به عنوان Discriminator، تمرکز بیشتر روی جزئیات لوکال و فرکانس بالای تصویر است. در مسائل تولید تصویر توابع هزینه معمول مثل MSE در ثبت فرکانس‌های پایین مانند رنگبندی کلی و ساختار اجسام خوب عمل می‌کنند اما تاری به وجود می‌آورند. شبکه تمایزدهنده وظیفه دارد این تاری را با مجبور کردن مولد به تولید لبه‌های تیز و بافت‌های واقعی جبران کند. همچنین PatchGAN به جای آنکه کل تصویر را با یک لیبیل واقعی یا فیک بسنجد، تصویر را تکه‌های کوچکتر تقسیم می‌کند و درباره واقعی بودن هر بخش نظر می‌دهد. این کار تعداد پارامترهای شبکه را کمتر می‌کند سرعت را بالا می‌برد. علاوه بر آن این معماری فرض می‌کند که پیکسل‌های

تصویر تنها به پیکسل‌های همسایه خود وابسته هستند و بنابراین استقلال بافت در نقاط دور از هم را مدل‌سازی می‌کند.

۴. در مدل GAN استاندارد تمایزدهنده تنها تلاش می‌کند تا تصاویر تولیدشده در همان لحظه توسط مولد را تشخیص دهد. در Cycle GAN یک بافر ایجاد می‌شود که ۵۰ تصویر تولیدشده اخیر را در خود ذخیره می‌کند و در هر مرحله از آموزش تمایزدهنده با ترکیبی از تصاویر جدید و تصاویری که به صورت تصادفی از این بافر انتخاب شده‌اند، آموزش می‌بیند. این مکانیزم برای حل مشکل ناپایداری در آموزش و نوسان مدل طراحی شده است. بدون وجود بافر مولد دائماً تغییر می‌کند تا تمایزدهنده فعلی را فریب دهد. پس از آن D خود را با تغییرات جدید G وفق می‌دهد. این چرخه ممکن است باعث شود G رفتارهای قبلی خود را فراموش کند و مدل بین چند حالت خاص دائماً نوسان کند. وجود بافر باعث می‌شود تمایزدهنده تنها به نسخه فعلی مولد اورفیت نشود بلکه یاد بگیرد تمام تصاویری که مولد در گذشته نزدیک تولید کرده را نیز تشخیص دهد. این کار مولد را مجبور می‌کند تصاویری بسازد که در طول زمان کیفیت پایداری داشته باشند و نه فقط تمایزدهنده را به صورت لحظه‌ای فریب دهند.

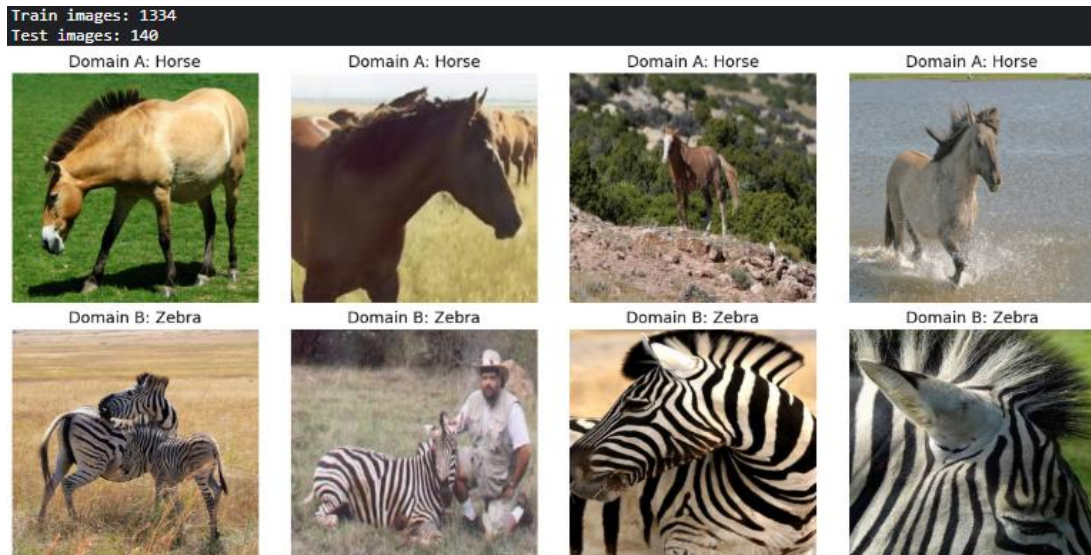
زیربخش سوم: محدودیت‌های مدل

۱. Cycle GAN در وظایفی که شامل تغییر بافت و رنگ هستند بسیار خوب عمل می‌کند اما در وظایفی که نیازمند تغییرات اساسی و هندسی بزرگ است شکست می‌خورد. مثلاً در تبدیل دو حیوان سگ و گربه با وجود شباهت‌های میان این دو اما ساختار صورت و شکل گوش و پوزه‌انی دو از نظر هندسی متفاوت است و این مدل نمی‌تواند تغییرات شکل صورت را به خوبی اعمال کند و صرفاً بافت گربه را روی صورت سگ تصویر می‌کند. دلیل اصلی این ضعف سختگیری بیش از حد قید سازگاری چرخه بر روی موقعیت مکانی پیکسل‌هاست. ترم‌های موجود در زیان سازگاری چرخه‌ای پیکسل به پیکسل عمل می‌کنند. برای مثال اگر مولد G بخواهد شکل هندسی را تغییر دهد، مولد برگشتی F کار بسیار سختی دارد تا جای دقیق پیکسل‌های اولیه را حدس بزند و آن‌ها را سر جای خودشان برگرداند. به عبارت دیگر هرگونه تغییر هندسی بزرگ ریسک خطای بازسازی را به حد زیادی بالا می‌برد. راه‌حل ساده مدل برای کمینه کردن خطای سازگاری چرخه‌ای این است که اصلاً تغییرات اساسی اعمال نکند و به ساختار هندسی و مکانی اشیاء دست نزند و به جای آن به موارد کم‌هزینه‌تری مانند بافت و رنگ پیکسل‌ها توجه کند.

بخش دوم: پیاده‌سازی مدل

زیربخش اول:

در این بخش از دادگان horse2zebra استفاده می‌کنیم. نمونه‌هایی از تصاویر این مجموعه به صورت زیر است:



شکل ۱. نمونه‌هایی از دادگان هر دو کلاس

زیربخش دوم:

با توجه به جزئیات ذکر شده در رابطه با پیاده‌سازی Generator و Discriminator به پیاده‌سازی آن می‌پردازیم. در ادامه از ابعاد پیشنهادی 128×128 استفاده می‌کنیم. خلاصه پیاده‌سازی مدل به صورت زیر است:

DISCRIMINATOR SUMMARY		
Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 64, 64]	3,136
LeakyReLU-2	[-1, 64, 64, 64]	0
Conv2d-3	[-1, 128, 32, 32]	131,200
InstanceNorm2d-4	[-1, 128, 32, 32]	0
LeakyReLU-5	[-1, 128, 32, 32]	0
Conv2d-6	[-1, 256, 16, 16]	524,544
InstanceNorm2d-7	[-1, 256, 16, 16]	0
LeakyReLU-8	[-1, 256, 16, 16]	0
ZeroPad2d-9	[-1, 256, 17, 17]	0
Conv2d-10	[-1, 512, 16, 16]	2,097,664
InstanceNorm2d-11	[-1, 512, 16, 16]	0
LeakyReLU-12	[-1, 512, 16, 16]	0
ZeroPad2d-13	[-1, 512, 17, 17]	0
Conv2d-14	[-1, 1, 16, 16]	8,193
Total params: 2,764,737		
Trainable params: 2,764,737		
Non-trainable params: 0		

شکل ۲. خلاصه پیاده‌سازی Discriminator

=====		
GENERATOR SUMMARY		
=====		
Layer (type)	Output Shape	Param #
=====		
ReflectionPad2d-1	[-1, 3, 134, 134]	0
Conv2d-2	[-1, 64, 128, 128]	9,472
InstanceNorm2d-3	[-1, 64, 128, 128]	0
ReLU-4	[-1, 64, 128, 128]	0
Conv2d-5	[-1, 128, 64, 64]	73,856
InstanceNorm2d-6	[-1, 128, 64, 64]	0
ReLU-7	[-1, 128, 64, 64]	0
Conv2d-8	[-1, 256, 32, 32]	295,168
InstanceNorm2d-9	[-1, 256, 32, 32]	0
ReLU-10	[-1, 256, 32, 32]	0
ReflectionPad2d-11	[-1, 256, 34, 34]	0
Conv2d-12	[-1, 256, 32, 32]	590,080
InstanceNorm2d-13	[-1, 256, 32, 32]	0
ReLU-14	[-1, 256, 32, 32]	0
ReflectionPad2d-15	[-1, 256, 34, 34]	0
Conv2d-16	[-1, 256, 32, 32]	590,080
InstanceNorm2d-17	[-1, 256, 32, 32]	0
ResidualBlock-18	[-1, 256, 32, 32]	0
ReflectionPad2d-19	[-1, 256, 34, 34]	0
Conv2d-20	[-1, 256, 32, 32]	590,080
InstanceNorm2d-21	[-1, 256, 32, 32]	0
ReLU-22	[-1, 256, 32, 32]	0
ReflectionPad2d-23	[-1, 256, 34, 34]	0
Conv2d-24	[-1, 256, 32, 32]	590,080
InstanceNorm2d-25	[-1, 256, 32, 32]	0
ResidualBlock-26	[-1, 256, 32, 32]	0
ReflectionPad2d-27	[-1, 256, 34, 34]	0
Conv2d-28	[-1, 256, 32, 32]	590,080
InstanceNorm2d-29	[-1, 256, 32, 32]	0
ReLU-30	[-1, 256, 32, 32]	0
ReflectionPad2d-31	[-1, 256, 34, 34]	0
Conv2d-32	[-1, 256, 32, 32]	590,080
InstanceNorm2d-33	[-1, 256, 32, 32]	0
ResidualBlock-34	[-1, 256, 32, 32]	0
ReflectionPad2d-35	[-1, 256, 34, 34]	0
Conv2d-36	[-1, 256, 32, 32]	590,080
InstanceNorm2d-37	[-1, 256, 32, 32]	0
ReLU-38	[-1, 256, 32, 32]	0
ReflectionPad2d-39	[-1, 256, 34, 34]	0
Conv2d-40	[-1, 256, 32, 32]	590,080
InstanceNorm2d-41	[-1, 256, 32, 32]	0
ResidualBlock-42	[-1, 256, 32, 32]	0
ReflectionPad2d-43	[-1, 256, 34, 34]	0
Conv2d-44	[-1, 256, 32, 32]	590,080
InstanceNorm2d-45	[-1, 256, 32, 32]	0
ReLU-46	[-1, 256, 32, 32]	0
ReflectionPad2d-47	[-1, 256, 34, 34]	0
Conv2d-48	[-1, 256, 32, 32]	590,080
InstanceNorm2d-49	[-1, 256, 32, 32]	0
ResidualBlock-50	[-1, 256, 32, 32]	0
ReflectionPad2d-51	[-1, 256, 34, 34]	0
Conv2d-52	[-1, 256, 32, 32]	590,080
InstanceNorm2d-53	[-1, 256, 32, 32]	0
ReLU-54	[-1, 256, 32, 32]	0
ReflectionPad2d-55	[-1, 256, 34, 34]	0
Conv2d-56	[-1, 256, 32, 32]	590,080
InstanceNorm2d-57	[-1, 256, 32, 32]	0
ResidualBlock-58	[-1, 256, 32, 32]	0
ConvTranspose2d-59	[-1, 128, 64, 64]	295,040
InstanceNorm2d-60	[-1, 128, 64, 64]	0
ReLU-61	[-1, 128, 64, 64]	0
ConvTranspose2d-62	[-1, 64, 128, 128]	73,792
InstanceNorm2d-63	[-1, 64, 128, 128]	0
ReLU-64	[-1, 64, 128, 128]	0
ReflectionPad2d-65	[-1, 64, 134, 134]	0
Conv2d-66	[-1, 3, 128, 128]	9,411
Tanh-67	[-1, 3, 128, 128]	0
=====		
Total params: 7,837,699		
Trainable params: 7,837,699		
Non-trainable params: 0		

شکل ۳. خلاصه پیاده‌سازی Generator

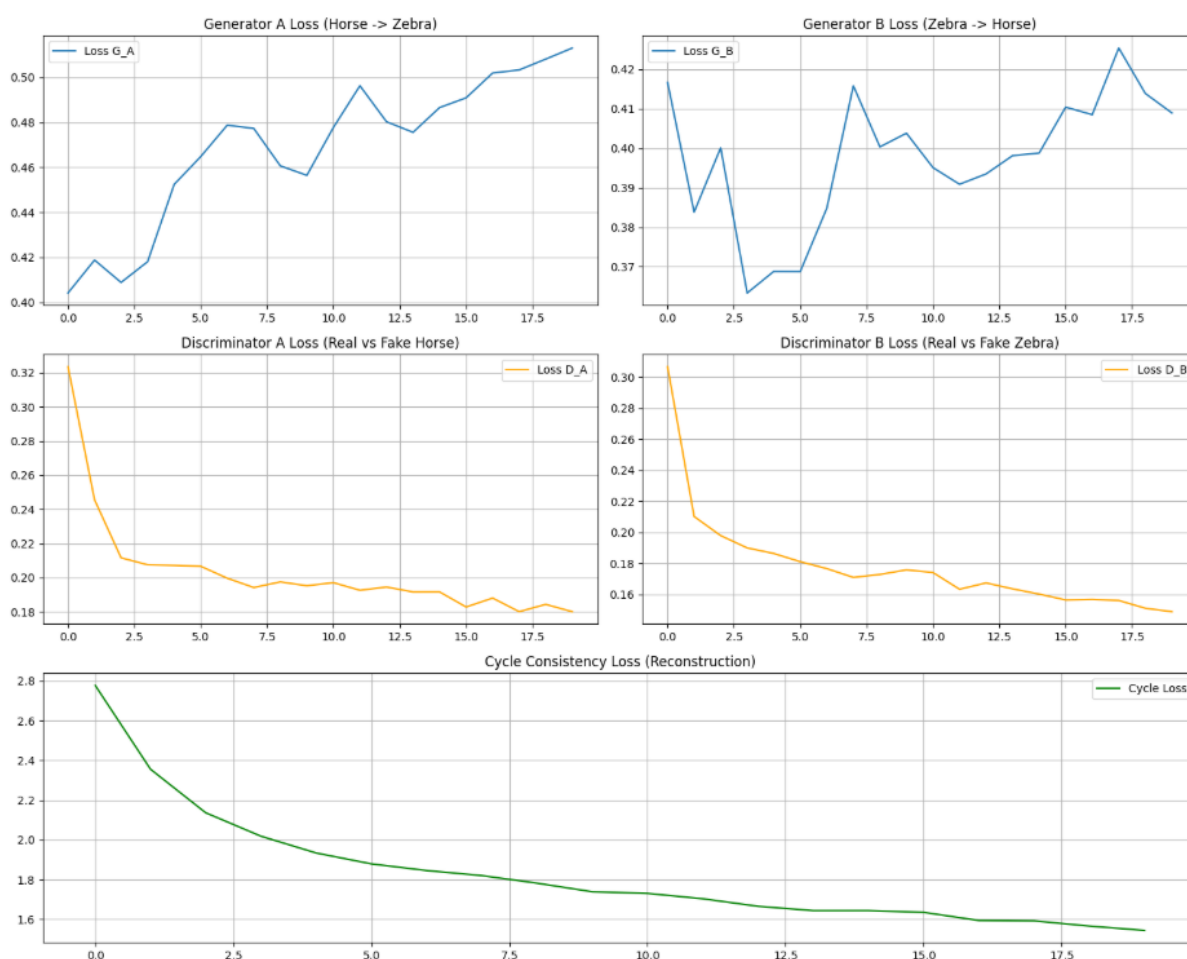
زیربخش سوم:

براساس مقاله مقادیر هایپرپارامترها را به صورت زیر تنظیم می کنیم. در این بخش ضریب خطای همانی را صفر و در بخش بعد مطابق مقاله برابر با ۵ در نظر می گیریم:

```
EPOCHS = 20  
BATCH_SIZE = 1  
LR = 0.0002  
B1 = 0.5  
B2 = 0.999  
LAMBDA_CYC = 10.0  
LAMBDA_ID = 0.0  
HEIGHT, WIDTH = 128, 128
```

شکل ۴. مقادیر هایپرپارامترها

نمودارهای زیان خواسته شده به صورت زیر می باشند:



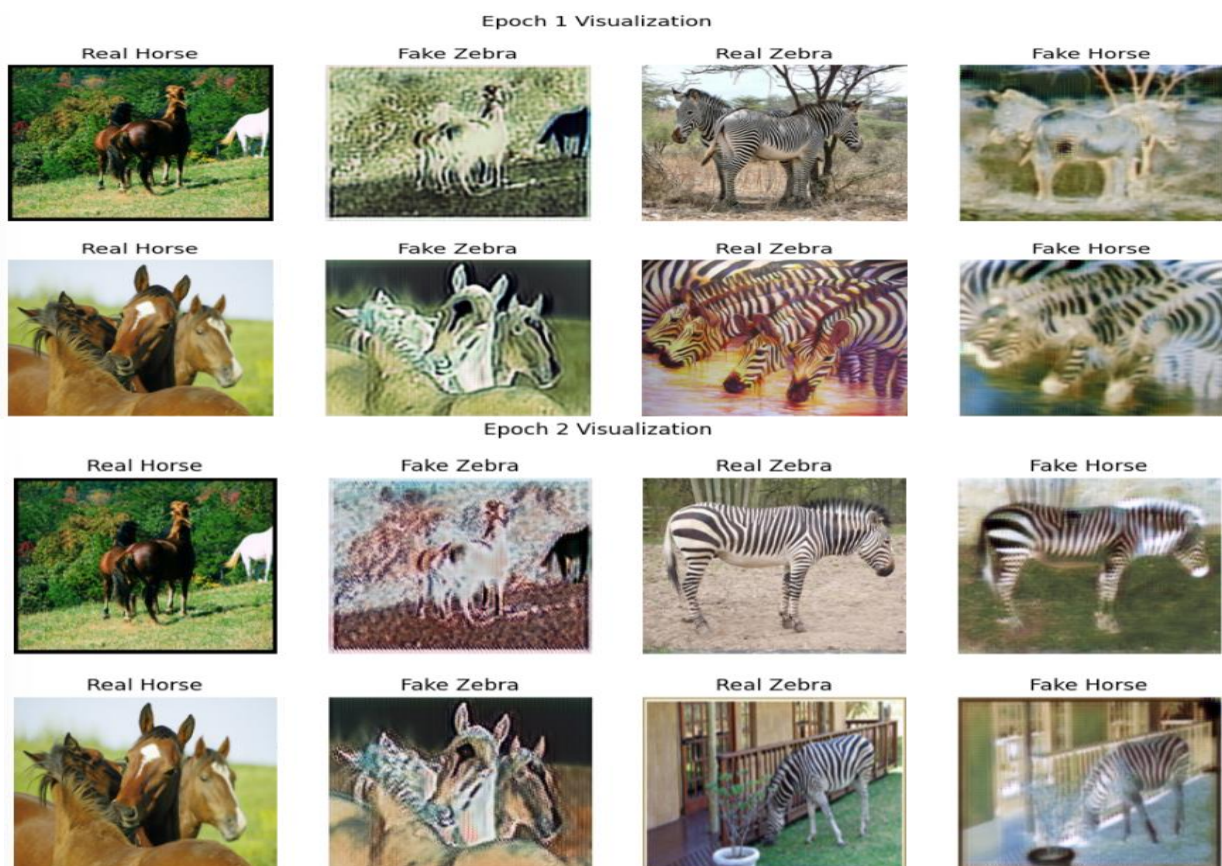
شکل ۵. نمودارهای زیان در هنگام آموزش

می‌توان گفت که مهمترین و قابل استنادترین نمودار برای بررسی نمودار مربوط به زیان سازگاری چرخه‌ای است. مشاهده می‌شود که این نمودار به آرامی و پیوستگی روند کاهشی داشته و نشان‌دهنده این است که مدل در حال یادگیری است و دارد یاد می‌گیرد تا تصاویر را به دامنه دیگر ببرد و برگرداند بدون اینکه محتوا را خراب کند. این به این معناست که ساختار کلی اسب حفظ می‌شود.

نمودار تمایزدهنده‌ها نشان‌دهنده این است که به سرعت یاد گرفته‌اند تا تصاویر تولیدی اولیه را که بیشتر نویز بوده‌اند را به خوبی از تصاویر واقعی تشخیص دهند. پایین ماندن خطای یعنی تمایزدهنده‌ها به نسبت قوی هستند و به راحتی نمی‌توان آن‌ها را فریب داد. این باعث می‌شود تا بر مولدها برای تولید تصاویر بهتر فشار وارد شود.

نمودار مولدها نیز نشان می‌دهد که مولد گورخر به اسب نوسانی است اما تقریباً ثابت مانده و مولد اسب به گورخر در حال افزایش خطاست. این نشان‌دهنده این است که مولد دوم هنوز نتوانسته تمایزدهنده گورخر را فریب دهد اما افزایش خطا نشان‌دهنده این است که تلاش در جریان است و مدل ساکن نشده‌است. به طور کلی احتمالاً با آموزش به اندازه ۲۰۰ اپیاک ذکر شده در مقاله نتایج بسیار بهتر خواهند شد.

خروجی‌های مدل به صورت زیر است:



Epoch 3 Visualization

Real Horse



Fake Zebra



Real Zebra



Fake Horse



Real Horse



Fake Zebra



Real Zebra



Fake Horse



Epoch 9 Visualization

Real Horse



Fake Zebra



Real Zebra



Fake Horse



Real Horse



Fake Zebra



Real Zebra



Fake Horse



Epoch 10 Visualization

Real Horse



Fake Zebra



Real Zebra



Fake Horse



Real Horse



Fake Zebra



Real Zebra



Fake Horse



Epoch 11 Visualization

Real Horse



Fake Zebra



Real Zebra



Fake Horse



Real Horse



Fake Zebra



Real Zebra



Fake Horse



Epoch 18 Visualization

Real Horse



Fake Zebra



Real Zebra



Fake Horse



Real Horse



Fake Zebra



Real Zebra



Fake Horse



Epoch 19 Visualization

Real Horse



Fake Zebra



Real Zebra



Fake Horse



Real Horse



Fake Zebra

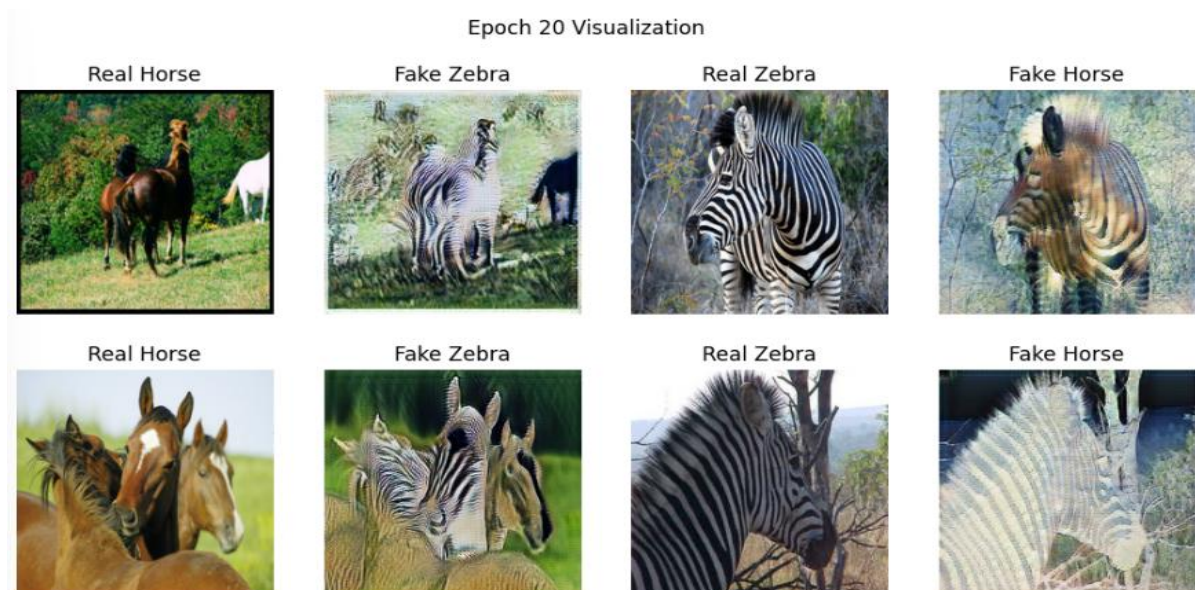


Real Zebra



Fake Horse



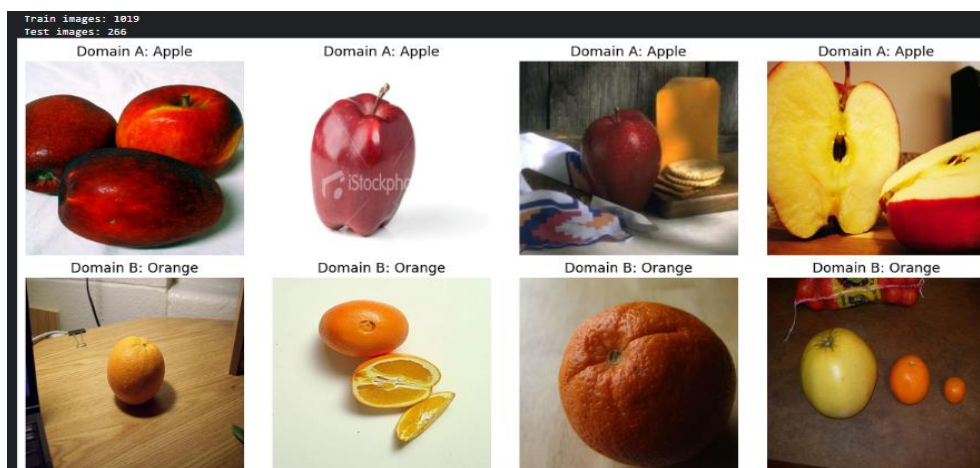


شکل ۶. تصاویر تولیدشده توسط مدل و تصاویر واقعی

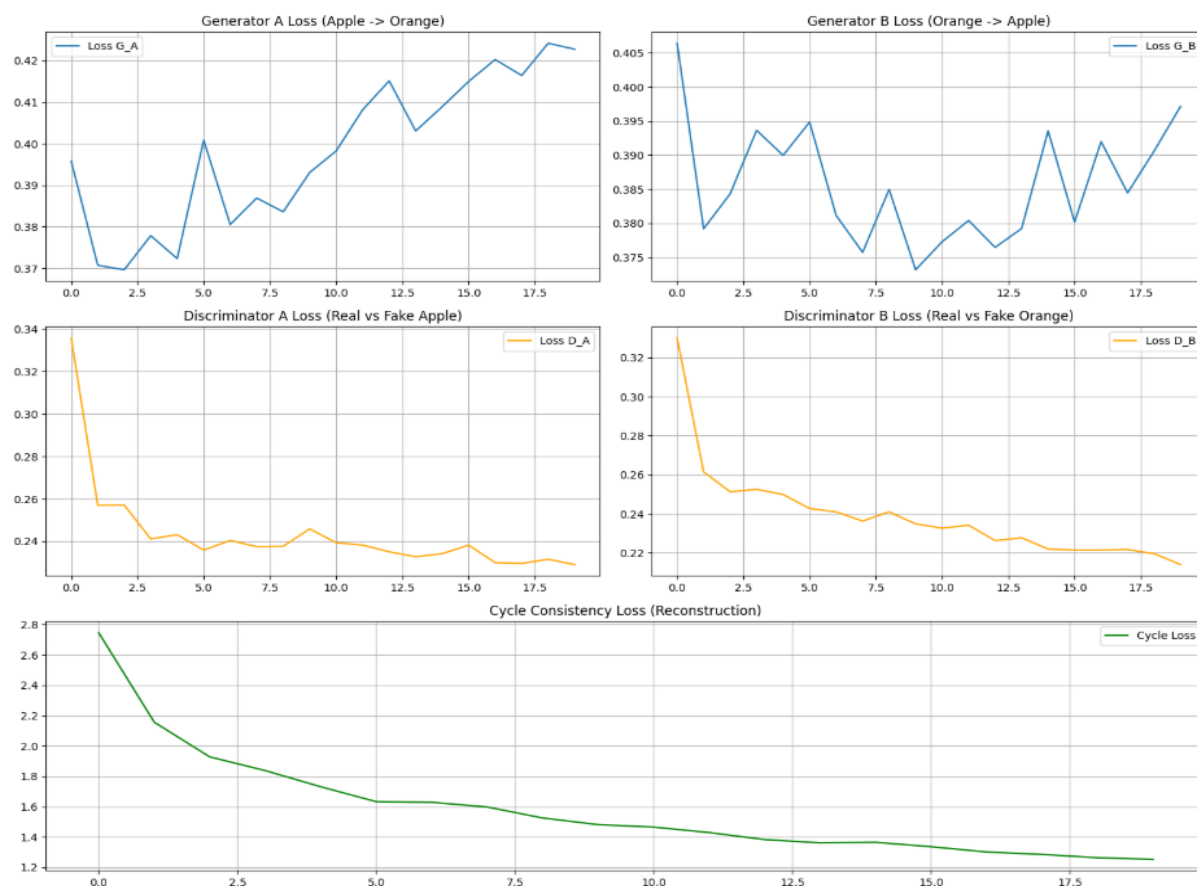
در ایپاک‌های آغازین خروجی‌ها تقریباً نویز خالص هستند و تصاویر دارای یک الگوی شطرنجی هستند که به دلیل استفاده از ConvTranspose2d در ساختار مدل است. در اینجا مدل صرفاً شروع به تلاش کردن کرده است. در ایپاک‌های میانی می‌توان مشاهده کرد مدل دارد رنگ‌های گورخر و اسب را یاد می‌گیرد و کم‌کم آن را اعمال کند. در ایپاک‌های پایانی نیز تلاش مدل برای تبدیل اسب به گورخر و برعکس کاملاً مشهود است اما پس‌زمینه نیز اندکی دچار تغییر شده که به دلیل عدم استفاده از زیان همانی در این بخش است. البته با استفاده از زیان همانی از ابتدا ممکن بود کلاً تبدیلی نداشته باشیم و یک استراتژی مناسب می‌تواند این باشد که تا مثلاً ۵۰ یا ۷۰ ایپاک بدون این لاس مدل را ترین کنیم و پس از آن ضریب این زیان را کم کم افزایش دهیم تا هم تبدیل به درستی انجام شود و هم پس‌زمینه بازیابی شود. به طور کلی مشخصاً مدل در حال یادگیری است.

زیربخش چهارم:

در این بخش از دادگان Apple2Oranges استفاده می‌کنیم و ضریب خطای همانی را مطابق با مقاله برابر با ۵ در نظر می‌گیریم.



شکل ۷. نمونه‌هایی از دادگان دو کلاس



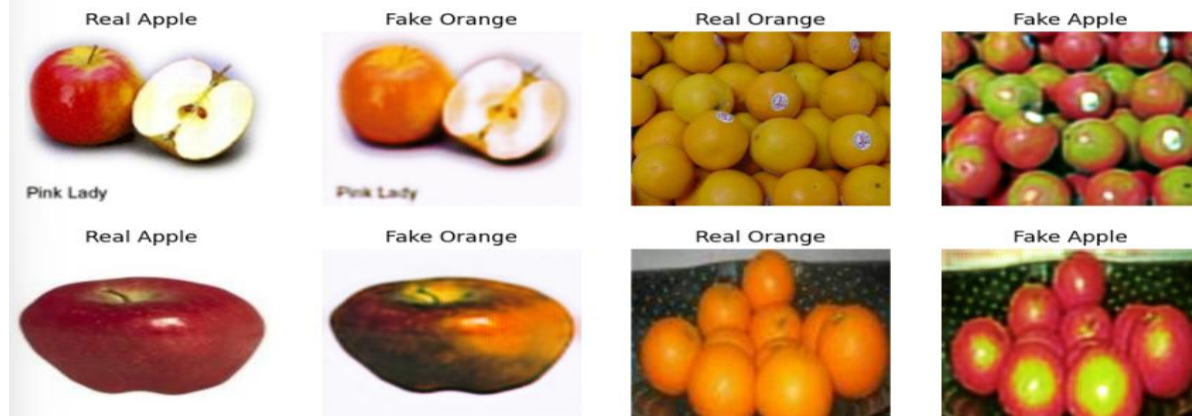
شکل ۸. نمودار زیان‌های خواسته شده

نمودارهای زیان این بخش نیز تقریباً مثل بخش قبل هستند و تا حد خوبی توضیحات مشابه است. در اینجا نیز یادگیری در حال اتفاق افتادن است و تمایزدهنده‌ها تا اینجا قدرتمندتر از مولدها هستند.

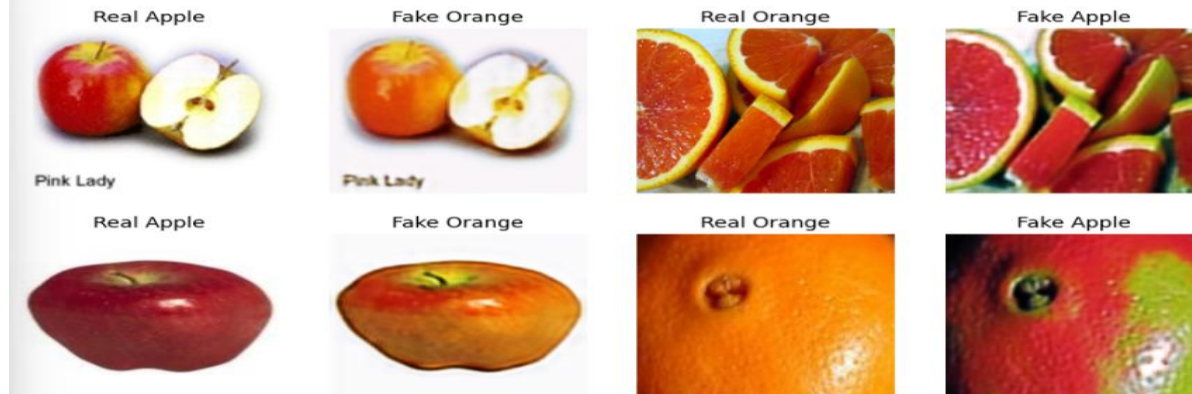
Epoch 1 Visualization



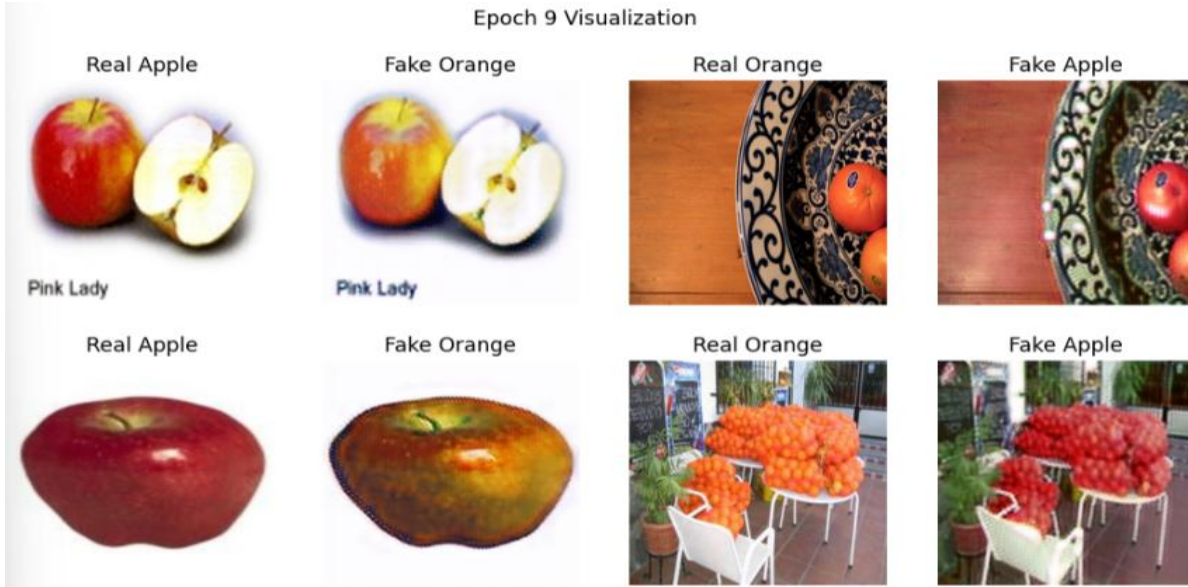
Epoch 2 Visualization



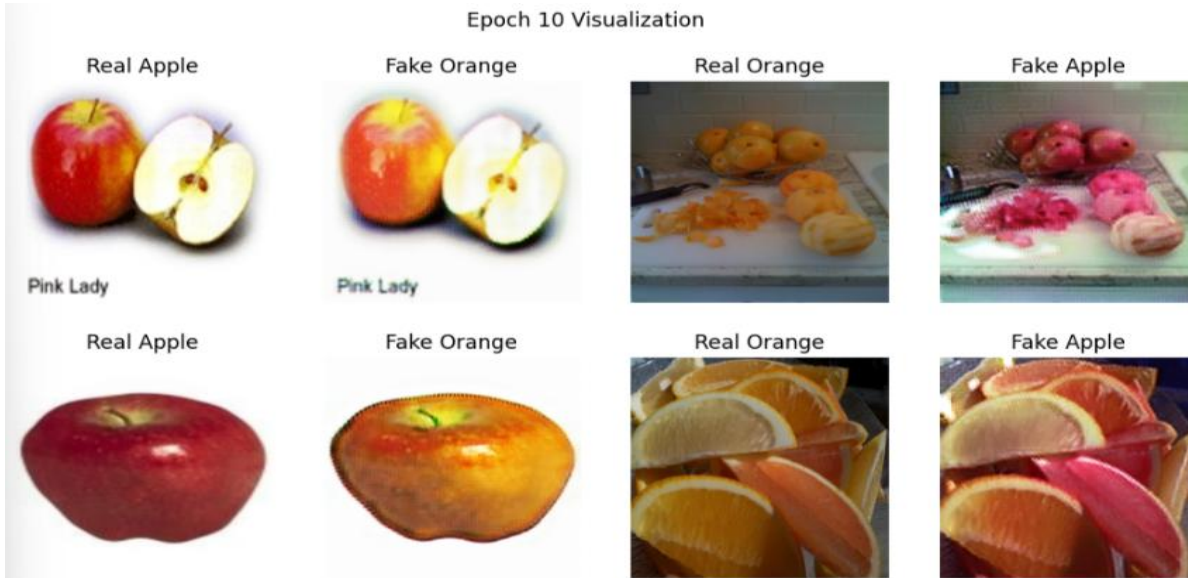
Epoch 3 Visualization



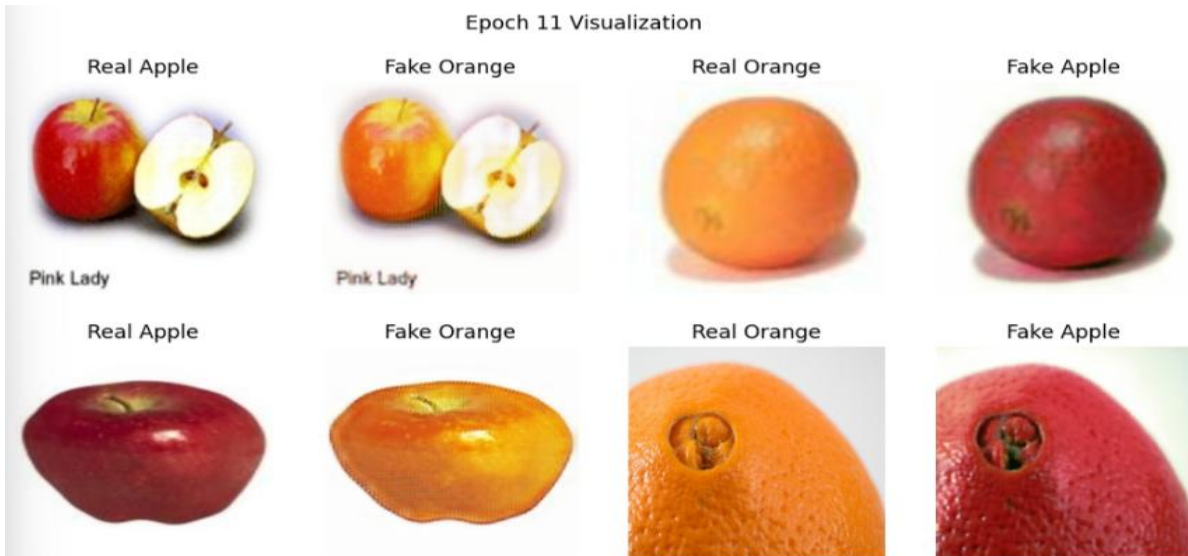
Epoch 9 Visualization



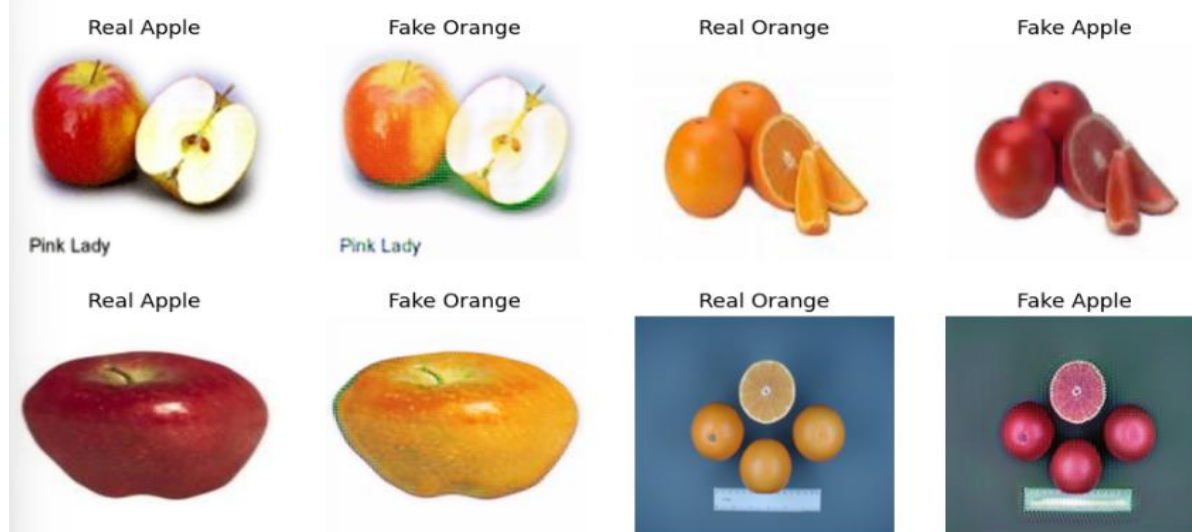
Epoch 10 Visualization



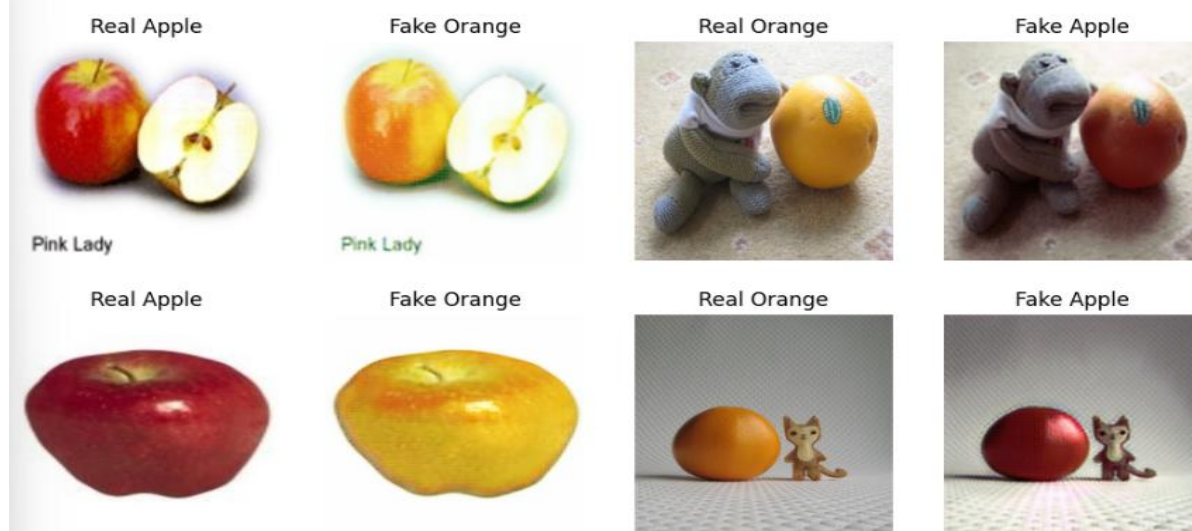
Epoch 11 Visualization



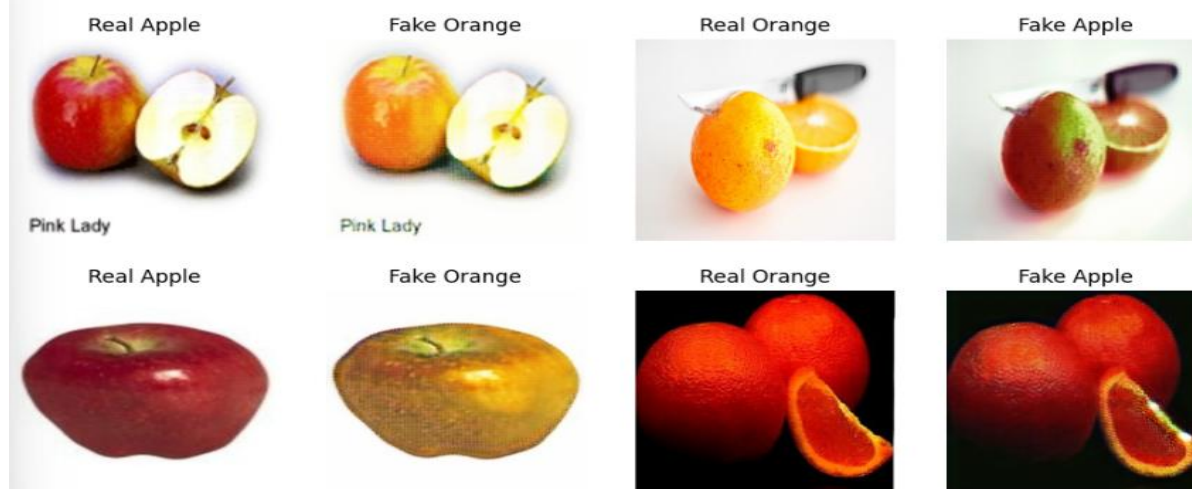
Epoch 18 Visualization



Epoch 19 Visualization



Epoch 20 Visualization



شکل ۹. تصاویر تولیدشده توسط مدل به همراه تصاویر واقعی

اینبار با وجود زیان همانی می‌توان مشاهده کرد که این زیان باعث شده پس‌زمینه تصویر کم‌تر دست بخورد و مدل بیشتر روی خود میوه تمرکز کند تا اطراف آن. همچنین مشخص است که مدل در حال یادگیری دامنه رنگی و تغییر آن است اما محدودیت هندسی Cycle GAN نیز در اینجا واضع‌تر شده چرا که مثلاً در تصاویر پرتقال ساخته‌شده، این تصاویر شکل هندسی سیب را دارند و مدل تنها می‌تواند رنگ و بافت آن‌ها را به پرتقال واقعی نزدیک کند.

- 1: Isola, P., Zhu, J. Y., Zhou, T., & Efros, A. A. (2017). Image-to-image translation with conditional adversarial networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2017) (pp. 1125–1134).
- 2: Zhu, J. Y., Park, T., Isola, P., & Efros, A. A. (2017). Unpaired image-to-image translation using cycle-consistent adversarial networks. In Proceedings of the IEEE International Conference on Computer Vision (ICCV 2017) (pp. 2223–2232).